Lib GAA document

Ziyou Wang

May 10, 2026

Abstract

Document for lib GAA, get it here.

Keywords: *C++, geodesy, photogrammetry, gnss*

CONTENTS

1. INTRODUCTION

The lib GAA (Geomatics Algorithms Application) provides a series of algorithms widely using in domain of geomatics (undergraduate level) and corresponding utility.

2. MOTIVATION

Provide implementation of geomatics algorithms for undergraduate level students in major of geomatics (surveying and mapping). Its features are not unique, but they may separate in serval libraries and their code may hard to understand for beginners. As a result, I develop this library, easy to use, easy to understand, but not fast, not so correct.

3. NAMING RULES

Before starting, let me illustrate naming rules of GAA.

```
// -------------------- macros --------------------
// This is a public macro, you can use it
// snake_casing + all upper casing + prefix 'GAA_'
#define GAA_PP_STRIP_PARAM
// This is a private macro, use for your own risk
// snake_casing + prefix 'GAA_'
#define GAA_msvc_dll_patch
// This is design for internally use, it's not stable
// snake_casing + prefix 'gaa_'
#define gaa_assert

// -------------------- functions --------------------
// This is a public function or method
// snake casing
extern double deg2rad(double);
// This is a private function
// snake casing + prefix '_'
```

```cpp
double _assert_fail(...);
// -------------------- class/struct --------------------
// This is a public class
// snake_casing + initial letter upper casing
class Gauss_kruger_project;
// This is a private class
// snake_casing + prefix '_' and optional suffix '_t'
class _ellipsoid_data_section_t;
// typename inside type traits
// snake_casing
// type argument in template
// PascalCasing
template <class Unit> struct _quantity_traits {
  using unit_type = Unit;
  constexpr static unit_type unit{};
  using base_type = boost::units::quantity<unit_type>;
};
// -------------------- enum --------------------
// enum name and enum value
// snake_casing + each initial letter upper casing
enum Table_Storage_Flag : int {
  Tab_Unsupported = 0,
  Tab_String,
  Tab_Double,
  Tab_Integer,
  Tab_Bool,
  Tab_COUNT
};
// -------------------- constants --------------------
// read only objects
// snake_casing
extern double const rho0, rho1, rho2;
```

4. CORE

The `core` module serves as the base of other module. It provides

- customized assertion macro
- preprocessor macro (like Boost.PP)
- keyword argument extension
- convenient wrapper of 3rd party library (Eigen, Boost.Unit)
- API of other languages (R)
- more math function and constants
- domain specified utility
- io function of specified format (csv)
- and more ...

some of them are internally used, while others may be used as part of public API. In general, you can include what you want, that's fine.

4.1. features

- Table4.2
- csv4.3

4.2. Table

The class `Table`, imported by `<gaa/core/table.hpp>` is designed as a named column major data frame to store heterogeneous data in one object. You can

access column using `.at<>()` by index or name
add column (move only) using `.push_back()` with given name or automatically generated
iterate columns using `.columns()`
access row using `.row()` by index, it automatically returns `Table_row_ref` (read or write) or `Table_row_view` (read only)
insert meta info using `.meta_ioa()` with given name and value
access meta info using `.meta_at<>()` with its name
more container method like `.size() .clear() .empty() ...`
and others just view source code

To correctly access a column, you must give `.at<>()` a template argument candidate its type. Supported type are `Tab_double` `Tab_int Tab_bool Tab_string`, they are basically alias of `double int bool std::string`, but please use the previous which is automatically generated by macro. So do `.meta_at<>()`.

4.3. csv

The function `read_csv_auto` and its manual version `read_csv`, imported by `<gaa/core/io/csv.hpp>` is used to read a CSV file to a `Table`4.2. Function `read_csv_auto` automatically parse type of a column according to its content based on regular expression, while `read_csv` don't, so you must give it a vector of `Table_Storage_Flag`4.2. I recommend `read_csv_auto` unless it can't correctly parse type of a column.

5. GEODESY

5.1. features

- spatial reference system (earth ellipsoid)
- algorithm for geodetic solution problem
- projection

6. PHOTOGRAMMETRY

6.1. features

- collinearity condition equation
- space resection/intersection

7. GNSS

7.1. features

- cycle slip detection 7.2
- satellite position calculation 7.3

7.2. cycle slip detection

TODO

7.3. satellite position calculation

7.3.1. sv_pos_from_broadcast

Function `sv_pos_from_broadcast`, imported by header `<gaa/gnss/space/sv_pos_from_broadcast.hpp>`, is used to calculate position of satellite using parameters from broadcast ephemeris, and return in order of xyz. It accepts tow group of arguments, first is combined of parameters in broadcast ephemeris, and the second one is a time point `t` defining when the position is calculating. You can either give these arguments one by one or store them in a struct `Param_sv_pos_from_broadcast` except `t`. Also `Param_sv_pos_from_broadcast::from_table_row` help you to construct parameters from a `Table_row_ref`4.2, a typical pipeline is: convert RINEX file to csv file using `gnss_lib_py`, then read csv file using `read_csv_auto`4.3 to a `Table` object, finally construct `Param_sv_pos_from_broadcast` and invoke `sv_pos_from_broadcast`. Here are required parameters in broadcast ephemeris:

sqrtA Square root of the semi-major axis ($m^{\frac{1}{2}}$).
e Eccentricity (dimensionless).
t_oe Ephemeris reference time (seconds, GPS week seconds).
M_0 Mean anomaly at reference time (radians).
omega Argument of perigee (radians).
i_0 Inclination angle at reference time (radians).
IDOT Rate of inclination angle (radians/second).
C_us Amplitude of the sine harmonic correction term to the argument of latitude (radians).
C_uc Amplitude of the cosine harmonic correction term to the argument of latitude (radians).
C_is Amplitude of the sine harmonic correction term to the inclination angle (radians).
C_ic Amplitude of the cosine harmonic correction term to the inclination angle (radians).
C_rc Amplitude of the sine harmonic correction term to the geocentric distance (meters).
C_rs Amplitude of the cosine harmonic correction term to the geocentric distance (meters).
Omega_0 Longitude of ascending node at reference time (radians).
OmegaDot Rate of longitude of ascending node (radians/second).

8. BUILD

Build system of GAA is CMake. Just build as usual(`cmake .. && make -j16 && sudo make install`)

8.1. dependencies

Eigen3 (5.0.0 used in dev) headers
Boost (1.90.0 used in dev) headers + dll(optional)
CMake version $\geq$ 3.20

GAA provide option `GAA_USE_PRIVATE_3RD` to enable using bundle 3rd party headers in building and installing. With this option on, you don't need an existing Eigen3 or Boost.

Information

option `GAA_USE_PRIVATE_3RD` will also install Eigen3 and Boost(header only) to install prefix.